

《人工智能通识》

第十一讲: 围绕 Prompt 形成的 Agent 工程体系

——Skill、Harness、Agent Plugins

周天航

重质油全国重点实验室
中国石油大学 (北京) 未来能源学院

2026 年 9 月



先回顾: 第十讲的“能力选择指南”, 三句话还记得吗?

- Q1** 拿到需求先做什么? 填**需求卡** (六问), 再走**六步决策树**
- Q2** 决策树的输出是什么? 一个**能力组合**: Prompt + RAG + Function Call + ...
- Q3** 建的时候按什么顺序? **由轻到重**, 先跑通再加重, 每一级先问“够了吗”

第十讲帮你**选好了**能力组合。但选好之后, 一个新问题会很快出现:
写好的 Prompt、接好的工具、调好的 Agent, 怎样才能**复用、分发、并且越用越好**?
而不是每做一个项目都从一张白纸开始, 每换一个工具都重写一遍。

这是全课程的最后一讲: 把前十讲的零件, 装成一套**工程体系**。

情境: 实验室 AI 助手的 System Prompt, 一学期后变成了什么样

最初: 几百个 token

任务: 把这段中文译成英文
要求: 术语准确、保留单位

功能越加越多

一学期之后: 数千个 token

任务: 把这段中文译成英文	要求: 术语准确、保留单位
角色: 重质油实验室助手	文献综述方法 12 条
安全检查清单 30 项	DCS 解释口径 8 条
PDF / Excel 处理方法	工具规则与 Few-shot 示例
边界与约束 ...	...

红色: 本次任务真正需要的; 灰色: 与本次无关, 却每次都进入 Context

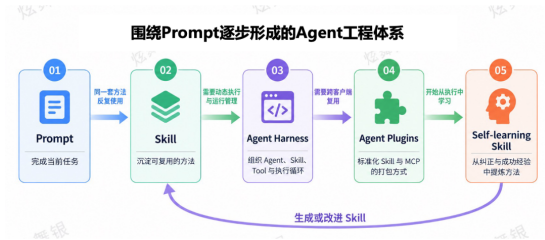
课堂互动①·举手投票

你有没有一段反复复制粘贴给 AI 的“万能开头”？

A. 没有 B. 有, 几十个字 C. 有, 几百个字 D. 有, 是一整个文档

选 C、D 的同学说一句: 这段开头里, 有多少内容是**每次都用得上的**?

今天的路线: 围绕 Prompt 逐步形成的五层工程体系



Prompt Skill

完成当前任务

把可复用的工作方法从大 Prompt 里拆出来, 按需加载

第三讲 本讲

Agent + Harness

Agent 动态决策; Harness 组织 Prompt、Skill、Tool 与执行循环

第七讲 + 本讲

Plugin 自我进化

把 Skill 和 MCP 标准化打包, 跨客户端分发
从执行过程中提炼经验, 反过来生成或修改 Skill

本讲
本讲

五层不是互相替代的几代技术,而是逐渐叠起来的工程层。

本讲学习目标

知识目标

说清 Prompt、Skill、Agent、Harness、Plugin 五层各自解决什么问题; 解释 Skill 的目录结构与三层渐进式加载; 知道 Agent Plugins 的统一打包格式和“自我进化”的受控流程。

能力目标

能把一份化工 SOP (如催化裂化装置开停车) 改写成一个 Skill 目录; 能为一组实验室工作方法设计一个可分发的 Plugin; 能为“让 Agent 修改 SOP”列出审核护栏。

素养目标

把“验证过的方法沉淀下来”当成习惯; 对 AI 自动生成的规程保持审核意识; 明确哪些操作必须经过人的确认。

学习建议: 每讲到一层, 先问“它把什么东西从 Prompt 里拿了出去, 放到了哪里”。

第一部分 · 简单任务, 从 Prompt 开始

Prompt 是我们在应用层与大模型交互的**基本方式**

无论外面的产品界面多么复杂, 模型每次真正接收到的, 最终仍然是一组 **token**。

用户的问题、系统指令、模板内容、示例 (Few-shot)、历史消息、输出要求



都以某种形式进入 Context, 再统一送给 LLM

研究发现: 换用更高级的模型带来的性能提升, 只有一**半**归功于模型本身,
另一半来自用户如何调整提示词。

第三讲讲过的四要素 (角色、指令、输入、输出格式) 和五个技巧, 今天全部还在。

边界清晰的简单任务, 写好 Prompt 通常就够了

你是一名行业研究员。
请分析下面这家公司的商业模式。
要求:

1. 说明客户是谁;
2. 说明产品和收入来源;
3. 分析主要成本与风险;
4. 使用普通读者能够理解的语言;
5. 不确定的信息必须明确标注。

一个例子

换成化工版, 只要改角色和要求:

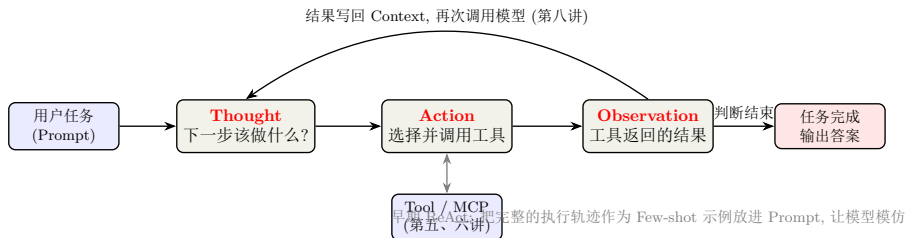
你是一名催化裂化装置工艺工程师。
请解读下面这份交接班记录。
要求:

1. 列出所有偏离正常范围的参数;
2. 给出每项偏离的可能原因;
3. 标注哪些判断需要现场核实。

Prompt Engineering 关注的, 就是怎样组织角色、背景、任务、步骤、示例、限制条件和输出格式。

判断标准: 一个任务能通过一次或少数几次模型调用稳定完成, 就先用 Prompt 解决。

任务变复杂: ReAct 让 Prompt 开始承载“怎么一步步做”



ReAct 通过 Prompt 引导模型在 **Thought** → **Action** → **Observation** 之间循环:
先判断下一步做什么, 再选择并调用工具; 运行时把工具结果作为新的
Observation 写回 Context;
模型基于更新后的 Context 再判断, 如此循环直到任务完成——这就是第七讲
Agent 的雏形。

ReAct 原论文: [arXiv 2210.03629](https://arxiv.org/abs/2210.03629)。

此时 Prompt 里装的, 已经不只是“这次要做什么”

内容	作用	值班助手里的例子
任务与要求	这一次要做什么	“解释三号反应器今晚的温升”
工作方法	这类事一般怎么做	DCS 异常解释口径 8 条
工具使用规则	何时调哪个工具、参数怎么填	查历史趋势先取 24 h, 再缩到 2 h
执行步骤	先做什么后做什么	先查联锁状态, 再查进料变化
Few-shot 示例	完整的执行轨迹, 让模型模仿	上个月两次成功排查的全过程
约束条件	什么不能做	不得建议修改设定值

第三讲说过产品级 Prompt 会长达几百行, 原因就在这张表里:
后五行都是“方法”而不是“任务”, 但它们都被写进了同一段 Prompt。

Agent 掌握的能力越多, 问题越明显



每种能力都有自己的一套方法

代码审查有一套方法, 数据分析有一套方法, PDF 处理又有另一套方法。

实验室助手也一样: 文献综述 12 条规则、安全检查 30 项清单、DCS 解释口径 8 条 ...

如果它们都长期放在 System Prompt 中, 那么无论当前任务是否需要, 每次模型调用它们都会进入 Context:

- ▶ 额外的 token 消耗 (第二讲: 按 token 计费);
- ▶ 模型要同时处理不同的规则和约束, 负担加重;
- ▶ 第八讲讲过的“上下文腐烂”: 无关内容越多, 越容易答偏。

Q: 让模型翻译一段班报, 为什么要让它同时“背着”30 项安全检查清单?

更重要的问题：验证过的方法散落在各处

那些已经**验证有效**的工作方法，开始散落在越来越长的 Prompt、代码和配置文件中：

改一处忘一处，新项目找不到，换个工具就得重写。

炼厂怎么做？ 没有哪座炼厂把全部装置的操作规程印在一张工单上；规程按装置、按工况分册，岗位上**需要哪本翻哪本**。

于是一个很自然的想法：

能不能把这个越来越臃肿的 Prompt **拆开**，把不同任务对应的工作方法**分别保存**，
只在**需要时再加载**进来？

这就是 Skill 的出发点 ⇒ 第二部分。先做一个小练习，感受一下“拆”。

找一找：这次任务真正需要哪几条？(2 分钟)

课堂互动②

实验室助手的 System Prompt 摘要如下，本次任务是：“把这段交接班记录翻译成英文，术语准确、保留单位”。

请圈出这一次用得上的条目，举手报数。

- | | |
|----------------------------|---------------------------|
| 1 角色：重质油实验室 AI 助手 | 5 实验室安全检查 30 项清单与判定标准 |
| 2 通用要求：术语准确，单位保留，不确定处标注 | 6 DCS 异常解释口径 8 条 |
| 3 文献综述方法 12 条 (检索、筛选、引用格式) | 7 PDF / Excel 处理方法与字段映射规则 |
| 4 工具使用规则：何时查文献库，何时读 DCS | 8 三个完整任务示例 (各约 800 token) |

大多数人会圈 1、2 两条——其余六条加起来可能有几千 token，却在每一次翻译时都进入 Context。

Q:如果第 3 ~ 8 条各自单独保存，助手要靠什么知道“这次该取哪一条”？带着这个问题进入 Skill。

第二部分 · 同一套方法反复使用, 把它沉淀成 Skill

Skill: 把**一套专业工作方法**组织成一个独立目录, 由 Agent 在需要时发现并加载

2025 年 10 月, Anthropic 正式推出 **Agent Skills**: 为 Agent 提供一种更具**可组合性、可扩展性和可移植性**的专业能力组织方式。

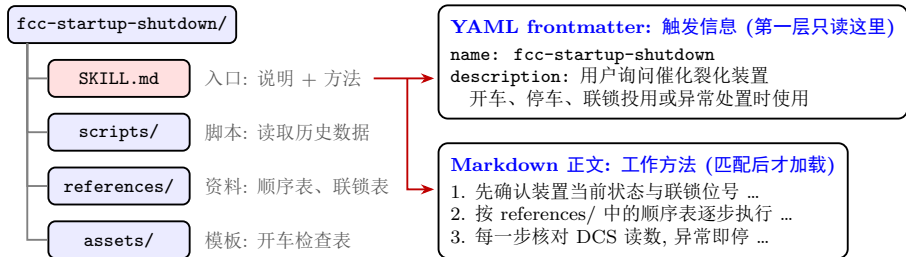
装什么 特定任务所需的 instructions (说明)、scripts (脚本)、resources (资源)

怎么描述 一个 SKILL.md: 这个 Skill**是什么、什么时候该用、具体怎么做**

怎么用 Agent 启动时只了解各 Skill 的名称和描述; 任务匹配时才加载完整内容

原本混杂在 Prompt 中的不同工作方法, 从此被**模块化**: 独立、可复用、按需加载。

一个 Skill 目录长什么样: 以“催化裂化装置开停车”为例



SKILL.md 是入口, 分两段: 顶部 YAML frontmatter 说“我是谁、何时用我”, 正文说“具体怎么做”;
其余三个目录是正文里会引用到的脚本、参考资料和模板——Skill 保存的**不只是一段 Prompt**。

SKILL.md 要回答的三个问题

```
---
name: fcc-startup-shutdown
description: 用户询问催化裂化装置
开车、停车、联锁投用或异常处置
步骤时使用；不用于日常班报解读。
---
# 催化裂化装置开停车
1. 先确认装置当前状态与联锁位号...
2. 按 references/sequence.md 的 顺
序
表逐步执行，每步核对 DCS 读数...
3. 读数异常即停，按 references/
emergency.md 处置，并提示人工确
认...
```

是什么 **name:** 一个稳定的短名字，便于被引用

何时用 **description:** 触发条件，也写清**不适用**的情形——这是 Agent 判断“要不要加载”的唯一依据

怎么做 正文: 步骤、规则、要引用哪些文件

description 写得含糊，要么该触发时不触发，要么翻译班报时也把它加载进来。

文件格式为示意，字段以所用平台的文档为准。

现场演示: 看几个真实的 Skill (3 分钟)

课堂互动③

打开公开仓库 github.com/anthropics/skills, 任选两个 Skill 目录:

- (1) 只读 SKILL.md 开头的 name 与 description, 猜它会在什么任务下被触发;
- (2) 再展开正文和子目录, 估一估: 在“只读描述”阶段, 有多大比例的内容**根本没有进入** Context。

两位同学口头汇报: 一个猜对了的, 一个猜偏了的。

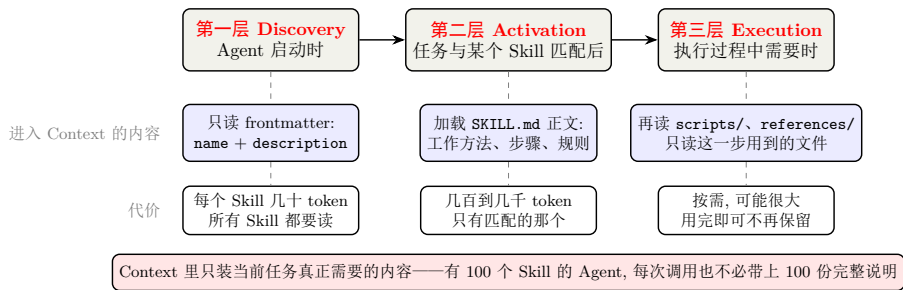
顺便看: Skill 的开放规范在 agentskills.io, 目录结构和 frontmatter 字段都写在那里。

看到的共同点 一个目录一件事; 描述短而具体; 正文像一份给新同事看的操作说明

看到的差异 有的只有一个 SKILL.md, 有的带脚本和几十页参考资料

Q: 为什么“描述短而具体”这么重要? 答案在下一页的加载机制里。

渐进式加载：三层，每层只取当前需要的



Agent 启动时不一次性加载所有 Skill 的完整内容, 只先了解名称和描述;
判断当前任务与某个 Skill 匹配时, 再加载它的完整 instructions; 执行中如需要, 再读脚本和参考资料。

三层渐进式加载, 再看一遍

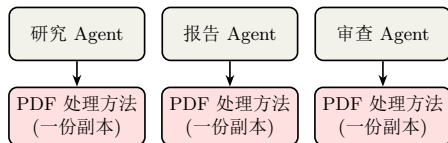


左侧是 Skill 目录里各组件的作用, 右侧是它们进入 Context 的先后顺序。

对照第八讲的四个动作:
Skill 就是把方法**写入**到窗口外, 匹配时再**选择**读回来——上下文工程在 Skill 上的具体实现。

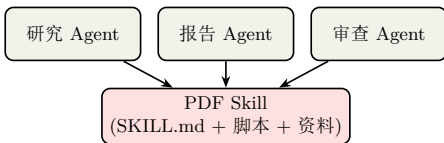
Skill 的另一个特点: 跨 Agent 复用

写进每个 SubAgent 的 System Prompt



三份内容一样, 三处分别维护 ❌

独立成一个 PDF Skill



一份内容, 多个 Agent 按需加载 ✔

一个 Agent 下面可能有多个 SubAgent: 研究 Agent、报告 Agent、审查 Agent 都要处理 PDF。

写进每个 SubAgent 的 System Prompt, 同一套内容就要重复编写和维护;

独立成一个 PDF Skill 后, 多个具备相应运行环境的 Agent 都可以在需要时使用同一个 Skill。

Skill 带来的变化, 三句话

1. 把工作方法从长 Prompt 中**拆出来, 独立保存**——不再散落在 Prompt、代码和配置里;
2. 同一个 Skill 可以被**不同 Agent 复用**——一份内容, 多处加载;
3. 通过**渐进式加载**, 只在需要时进入 Context——Skill 再多, 每次调用也不必全带上。

什么时候有 Skill 就够了, 不必上 Agent 循环?

如果任务只是**按照一套固定方法生成结果**, 不需要根据中间结果持续调整下一步——

比如“按实验室模板写一份安全检查报告”——由支持 Skill 的应用加载相应方法, 可能已经足够。

对照第十讲的决策树: **路径固定** → Prompt + Skill; **路径不固定** → 才需要下一部分的 Agent。

化工联结: 把一份 SOP 沉淀成 Skill, 各部分放哪里?

催化裂化开停车 SOP 的章节	在 Skill 里的位置	什么时候进入 Context
适用范围、何时启用本规程	frontmatter description	启动时 (Discovery)
操作步骤、注意事项、判停条件	SKILL.md 正文	任务匹配后 (Activation)
开停车顺序表、联锁逻辑表、应急预案	references/	执行到相应步骤时
开车前检查表、交接记录模板	assets/	需要填表时
读取 DCS 历史趋势的取数程序	scripts/	需要查数时

对比一下: 把整本 SOP 原样贴进 System Prompt, 等于让助手**翻译一段班报时也背着整本规程**;

拆成 Skill 之后, 只有真的在处理开停车问题时, 规程才进入 Context。

SOP 的内容本身一个字都不用改, 改变的只是“保存在哪里、什么时候被读”。

动手写: 给“实验室安全检查” Skill 写 frontmatter (3 分钟)

课堂互动④

只写两行: **name** (英文短横线命名) 和 **description** (一句话, 说清什么时候用、什么时候不用)。

写完与同桌交换, 用下面两个测试问题互相检验对方的描述。

测试 1 “帮我看看这周的安全检查表哪几项没填、哪几项不合格” 应当✓触发

测试 2 “帮我把安全培训 PPT 翻译成英文” 不应✗触发

加试 “通风橱风速 0.3 m/s 算合格吗” 你的描述能判断吗?

常见问题: 描述里只有“安全”两个字, 测试 2 就会误触发; 只写“检查表”, 加试就漏掉了。

description 是写给 Agent 看的“适用范围”条款, 和 SOP 第一章一样重要。

第三部分 · Agent Harness: 组织 Skill、Tool 和执行循环

有了 Skill, 下一个问题: Agent 怎么知道有哪些 Skill, 又怎么判断该加载哪一个?

这些工作**不是 Skill 自己完成的**, 而是由 **Agent Runtime (运行时)** 负责。

以 Claude Code 为例: 它的运行机制负责**发现**可用 Skill, 并在任务需要时把相应内容**加载进 Context**。

Skill 是一本本规程 放在书架上, 自己不会跳出来

Runtime 是那位值班的人 知道书架上有什么, 判断现在该翻哪一本

随着 Agent 越来越复杂, 这位“值班的人”要管的事也越来越多——这一部分讲它。

先回答: 什么时候才需要 Agent?

当任务**无法预先确定全部步骤**, 需要根据搜索结果、文件内容或 Tool 返回值**持续判断下一步**时, 就需要 Agent。

任务	步骤能否预先写死	用什么
按模板生成一份安全检查报告	能	Prompt + Skill
把 20 篇 PDF 的摘要翻译成中文	能 (循环固定)	Workflow + Skill
“三号反应器今晚温升为什么偏快?”	不能: 查完联锁才知道下一步查什么	Agent (ReAct 循环) + Skill

第七讲的 Agent 循环 (感知 → 决策 → 行动) 与 ReAct 的 Thought → Action → Observation 是同一件事;

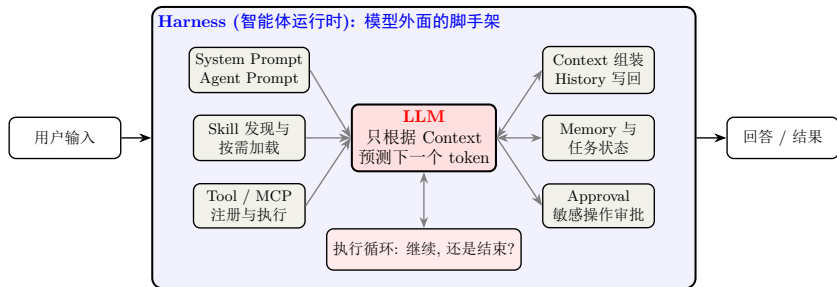
第十讲决策树里“路径是否固定”那一步, 判断的正是要不要它。

在模型外部, 需要一套运行系统负责这九件事

职责	在哪一讲见过
1 保存 System Prompt 和 Agent Prompt	第三讲
2 接收用户输入	
3 管理 Tool 和 Skill	第五、六讲 + 本讲
4 执行模型选择的 Tool	第五讲: 模型只“想”, 程序才“做”
5 把必要的执行结果写回 Context	第八讲: 写入与选择
6 再次调用模型	第七讲: 循环
7 保存 History、Memory 和任务状态	第八讲: 窗口外的记忆
8 对敏感操作实施 Approval (审批)	第五讲: 写操作留人确认
9 判断任务继续还是结束	第七讲: 何时停

这套围绕模型运行的**脚手架**, 就是 **Harness**。你会发现: 九件事里没有一件是模型自己做的。

Harness: 模型外面的脚手架



中间的 LLM 仍然只做一件事: 根据 Context 预测下一个 token。
Prompt、Skill、Tool、Context、Memory、Approval 和执行循环, 全部
由 Harness 组织和运行。

Harness 框架管理的组件



上半部分是 Context 组装：System Prompt、用户输入、历史、记忆、当前匹配的 Skill、工具定义 ...拼成本轮送给模型的上下文。

下半部分是运行管理：规划、状态、Tool 执行、审批、Shell、代码执行、SubAgent、循环。

Skill 是 Harness 管理的一部分；Harness 负责把上下文、能力和运行过程组织起来。

三者的分工, 各用一句话说清

Skill 解决的是	“怎样 保存 一套专业工作方法”
ReAct Agent 解决的是	“怎样根据任务和中间结果 动态决定下一步 ”
Harness 解决的是	“怎样把 Prompt、Skill、Tool、状态和执行循环真正 组织起来 ”

Harness 还要管理越来越多的东西: Skill、Tool、Memory、Context、Approval、Middleware 和执行循环 ...

第八讲的上下文工程 (写入、选择、压缩、隔离), 在工程上正是 Harness 里 “Context 组装” 那一格的工作。

Q: 一个 Agent 换了底层模型 (比如从 DeepSeek 换成 Qwen), Skill 和 Harness 哪些要跟着改?

化工类比: 工单、规程、值班工程师与中控室



中控室: 规程、数据、权限、记录都在这里汇合

Prompt	一张工单 / 一次口头交办: 这次做什么
Skill	一本本分册的标准操作规程 (SOP): 这类事怎么做
Agent	值班工程师: 看着读数决定 下一步查什么
Harness	中控室 + 岗位手册 + 交接 班制度: 谁能动哪个阀、查 什么数据、哪些操作要主任 签字、全部留记录

好的中控室不替工程师做判断, 但保证他**看得到该看的、动不了不该动的、做过的都有记录。**

判断题 (先举手, 再揭晓)

课堂互动⑤

1. Skill 会自己判断当前任务是否匹配, 并把自己加载进 Context。
2. 给 Agent 换上一套更好的 Harness, 模型的参数也随之变好了。
3. 只要用了 Skill, 就一定需要 ReAct 那样的 Agent 循环。

1. ✗ 发现与加载由 Runtime / Harness 完成; Skill 只是被动地“写清楚自己何时该被用”。
2. ✗ 参数在推理时不变。Harness 改变的是模型周围的组织方式: Context 更干净、工具更可靠、状态有记录。
3. ✗ 按固定方法生成结果的任务, 由支持 Skill 的应用加载方法即可; 只有路径不固定的任务才需要 Agent 循环。

三道题合起来是一句话: 模型没变, 变的是模型外面的东西——这句话本讲还会再出现两次。

Harness 里最容易被忽略的一格: Approval

值班助手可能做的操作	处理方式	理由
读取 DCS 历史趋势、查联锁状态	自动执行	只读, 可回溯
检索历史工况库、生成解释报告	自动执行, 标注来源	输出给人看, 人再判断
“建议下调反应温度设定值”	只能作为建议输出	第八讲的教训: 约束要在 Harness 里硬性保证
下发检修工单、通知班组	人工确认后执行	有外部影响, 不可撤回
修改 SOP 或 Skill 本身	人工审核 + 版本记录	第五部分专门讨论

第五讲说“写操作留人确认, 危险操作不注册”; 到了 Harness 这一层, 这条规则有了固定的位置:

审批不写在 Prompt 里求模型遵守, 而是由 Harness 在执行前拦下来。

第四部分 · Agent Plugins: 标准化 Skill 和 MCP 的打包方式

Harness 解决了运行时怎么组织; 新问题是: 封装好的 Skill 和 MCP, 怎样在**不同客户端**之间复用?

过去, 不同 Agent 产品往往有自己的**目录结构、配置方式和扩展机制**: 同一套 Skill 和 MCP 想在几个客户端里都能用, 开发者要针对每个产品分别适配。

你在 A 工具里写好的文献综述 Skill	换到 B 工具, 目录要重摆、配置要重写
-----------------------	----------------------

实验室配好的 DCS 数据 MCP	每个客户端各有一份配置, 改一处漏一处
-------------------	---------------------

第六讲的 MCP 统一了“模型怎么接工具”; 现在缺的是: **能力包本身怎么统一地装箱、发货。**

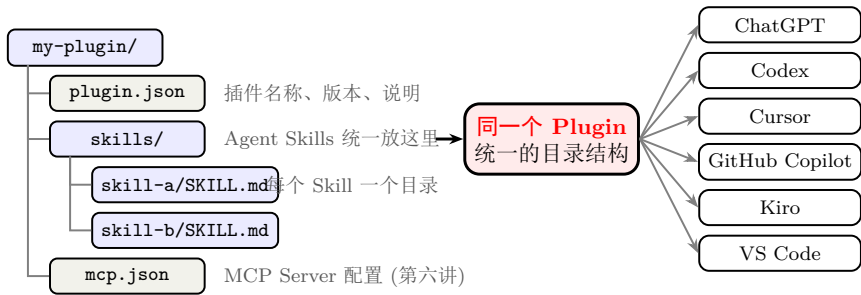
Agent Plugins 1.0.0: 一个开放、厂商中立的打包标准

时间	2026 年 8 月 6 日
发起	Vercel 联合 AWS、Anysphere、GitHub、Microsoft 和 OpenAI
首批支持的客户端	ChatGPT、Codex、Cursor、GitHub Copilot、Kiro、VS Code
它做了什么	没有重新定义 Skill 或 MCP, 而是在它们之上增加一层统一的打包标准
它没做什么	不规定 Skill 里写什么, 不规定 MCP Server 怎么实现

一个 Plugin 就是一个目录: 根目录放 `plugin.json`, Skill 统一放在 `skills/`, MCP Server 配置写在 `mcp.json`. 兼容标准的客户端按同一目录结构**发现和加载**。

标准刚发布, 各客户端的支持程度随版本变化, 以各自文档为准。

Plugin 的目录结构, 与跨客户端分发



左边是一个 Plugin 的目录树; 右边是同一个 Plugin 被多个客户端按同一规则发现和加载。

`skills/` 里就是第二部分的 Skill 目录, `mcp.json` 里就是第六讲的 MCP Server 配置——都是旧东西, 新的是箱子。

Skill、MCP、Plugin 各解决什么, 一张表分清

	解决什么问题	哪一讲	炼厂里对应什么
Skill	怎样把一套工作方法保存下来	本讲第二部分	标准操作规程 (SOP) 分册
MCP	怎样连接外部工具和数据	第六讲	DCS / LIMS 的数据接口
Plugin	怎样把这些能力按统一标准打包, 在不同客户端之间分发和复用	本讲第四部分	装置移交包 / 班组交接资料包

分层类比再说一遍:

SOP

一件事怎么做 (Skill)

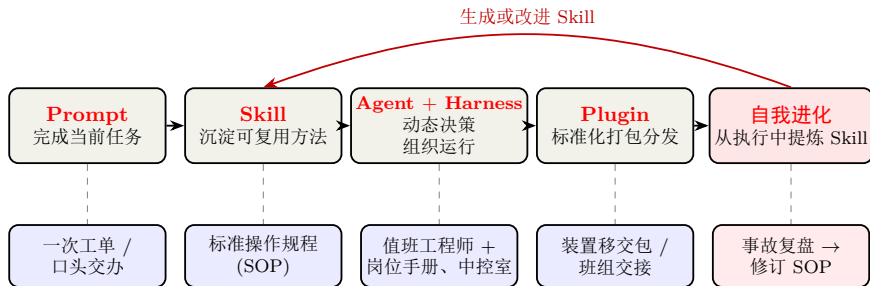
岗位手册 + 中控室

这个岗位有哪些规程、哪些权限、怎么运转 (Harness)

班组交接 / 装置移交包

把规程、图纸、接口清单整套交给下一班、下一套装置 (Plugin)

五层工程体系与炼厂规程体系, 逐层对照



上排是本讲的五层, 下排是你在炼厂里早就见过的东西。最右边那条“事故复盘 → 修订 SOP”的回路, 就是第五部分要讲的**自我进化**: 经验从执行中来, 回到规程里去。

化工联结: 实验室的一个 Skill 包

```
lab-plugin/  
  plugin.json  
  skills/  
    lab-safety-check/  
      SKILL.md  
      assets/checklist.xlsx  
  literature-review/  
    SKILL.md  
    references/  
  dcs-anomaly-explain/  
    SKILL.md  
    scripts/trend.py  
  mcp.json
```

Skill	装什么	要接的 MCP
安全检查	30 项清单、判定标准、报告模板	无 (纯方法)
文献综述	检索策略、筛选与引用规则	组内文献库 (第四讲的向量库)
DCS 异常解释	8 条解释口径、排查顺序、取数脚本	DCS 历史数据 (只读)

新同学入组: 装上这一个包, 三套方法和两个数据接口一次到位; 换了 AI 客户端: 同一个目录, 不用重摆。

mcp.json 里只放接口配置, 密钥与权限在各自的 MCP Server 端管理。

Plugin 对不同的人意味着什么

对你个人	Skill 的写法学一次就够; 换客户端时, 带走的是目录, 不是重写一遍的 Prompt
对课题组	组里验证过的方法沉淀成一个 Plugin, 版本统一; 师兄师姐毕业, 方法留下
对企业	装置级 SOP Skill 包随内部平台分发到各车间; 修订一次, 各处同步

这与第六讲 MCP 的价值是同一种: **标准化之后, 生态才能长起来。**
MCP 让工具可以被任何模型接入; Plugin 让 Skill + MCP 的组合可以被任何客户端加载。

Q: 一个课题组的 Plugin 里, 哪些 Skill 可以公开分享, 哪些必须留在组内? 依据是什么?

Skill 的开放规范与示例: agentskills.io; MCP 协议仓库:
github.com/modelcontextprotocol。

第五部分 · 自我进化: Agent 开始从执行过程中自动沉淀 Skill

前四层里, Skill 都是人预先写好的。最近, 这件事开始变化。

OpenClaw 2.0 的 **Self-learning Skill**: Agent 开始尝试从自己的执行过程中学习——

从哪里学 用户的纠正, 或一次成功完成的任务

学成什么 提炼出可复用的经验, 保存为新的 Skill, 或对已有 Skill 提出修改建议

给谁用 未来的 Session 可以发现并使用

对照炼厂: 这就是事故复盘 / 经验总结 → 修订 SOP 那条回路, 只是现在由 Agent 先起草。

OpenClaw 为开源项目, 仓库: github.com/openclaw/openclaw; 功能细节随版本变化。

一个例子：一次纠正，怎样变成一条方法

一个例子：Excel 字段映射

Agent 第一次处理某类 Excel 文件时，用了错误的字段映射。

用户指出：字段 A 必须**先转换格式**，再与字段 B 匹配。

提炼后的通用方法：

“处理此类 Excel 时，应先检查字段 A 的格式，完成转换后再与字段 B 进行关联。”

实验室版：LIMS 导出表

Agent 把 LIMS 导出的样品表与批次记录匹配时，一半样品没对上。

同学指出：样品编号列带**末尾空格和全角字符**，要先清洗再匹配。

提炼后的通用方法：

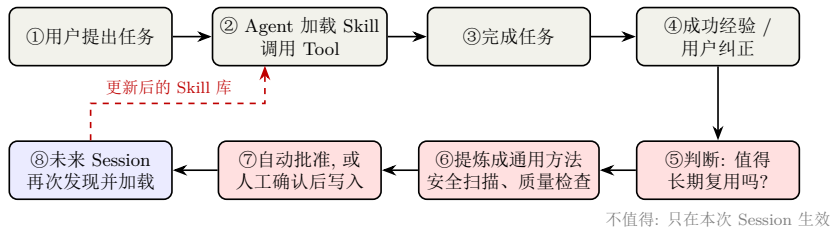
“处理 LIMS 导出表时，先对样品编号做去空格、全角转半角，再与批次号关联。”

过去 这次纠正只对当前 Session 有效，下次换个人问，同样的错再犯一遍

现在 Agent 先判断“这条经验有没有长期复用价值”：有，就生成新 Skill 或修改已有的 Excel Skill

下一次遇到相同任务，Agent 加载的是**更新后的 Skill**，不必再犯同样的错误。

于是, Prompt、Skill 与 Agent 之间形成了一个新的闭环



用户提出任务 → Agent 加载 Skill、调用 Tool → 完成任务 → 获得成功经验或
用户纠正

→ 提炼可复用的方法 → 生成或修改 Skill → 在未来 Session 中再次加载。

模型本身仍是原来的模型, 参数没有变。变的是: 有效经验被保存到了模型外部, 能在未来任务中重新进入 Context。

一张图看懂 OpenClaw 2.0 的 Self-learning

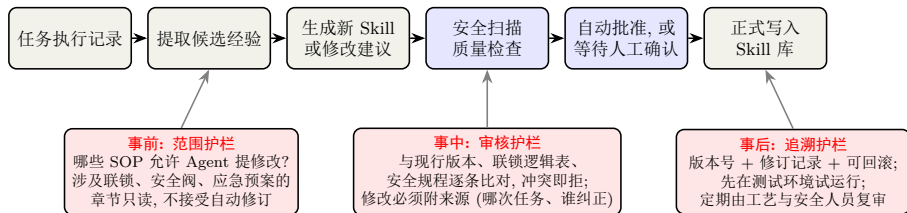


上排是本次任务: 提出、执行、完成、获得经验、判断是否值得长期复用。

中间的“Skill Workshop”把经验转成候选 Skill: 生成提案、安全扫描, 再按应用模式决定**自动写入**还是**等待人工确认**。

下排: 写入 Skill 库, 未来 Session 再次发现并使用。

这不是放任自流: Self-learning 被放进一套受控流程



上排是 OpenClaw 的六步: 记录 → 提取 → 生成 → 扫描检查 → 批准
或人工确认 → 写入。

下排是把它用到“让 Agent 改 SOP”时，化工人应该追加的三道护栏——下一页请你们自己补全。

小组讨论: 让 Agent 自己改“催化裂化开停车 SOP”, 你敢吗? (5 分钟)

课堂互动⑥

情境: 值班助手在三次开车任务中都发现, 规程第 4.2 步的等待时间偏短, 于是自动提出修改建议。

4 人一组, 给出至少 3 条审核护栏, 每条标明属于事前 / 事中 / 事后, 并说清“谁”负责。写在纸上, 选 2 组汇报。

可以从这些角度想:

- ▶ 什么样的修改建议一律不接受自动写入? (联锁、安全阀、应急预案 ...)
- ▶ 一条建议要附上什么证据才有资格进入审核? (来自哪几次任务、原始 DCS 数据、谁纠正的)
- ▶ 谁有权批准? 工艺、设备、安全三方要不要会签? 批准后先在哪里试运行?
- ▶ 改错了怎么回滚? 版本号和修订记录放在 Skill 的哪个位置?

评价标准: 护栏是否具体到“谁、在哪一步、看什么”; 有没有区分“自动可做”和“必须人签字”。

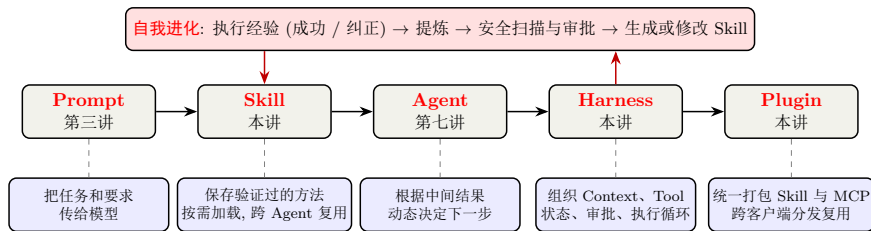
让 AI 改规程之前, 先让人签字: 负责任地沉淀经验

- ▶ Agent 提炼出的方法只是**候选**: 没有经过核对、试运行和人工确认的经验, 不写入共享 Skill 库;
- ▶ 每一条自动生成或修改的 Skill 都要**标注来源** (哪次任务、哪个模型、谁批准), 就像论文里标注引用;
- ▶ 涉及安全、合规、他人数据的操作, 不靠 Prompt 里的一句“请勿”约束模型, 而是在 Harness 的 Approval 里**硬性拦截**;
- ▶ 期末项目中, 请注明哪些 Skill 内容由 AI 生成、哪些经过了你们的验证——这是工程诚信, 也是学术诚信。

Self-learning 真正重要的地方, 不是 Agent 突然拥有了像人一样的学习能力, 而是 **Skill 的形成方式变了**: 从“人预先写好”, 变成“把这次是怎么做成的保存下来”。

Q: 炼厂修订一版 SOP 通常要走多少道审批? 你觉得其中哪几道对 Agent 生成的修订同样不能省?

本讲总结: 五层, 加一个闭环



模型仍然只是根据 Context 预测下一个 token; 变化的是模型周围这套工程体系

Prompt 指导一次任务; Skill 复用已有方法; Agent 动态决定下一步; Harness 组织整个执行过程;

Plugin 把 Skill 与 MCP 统一打包分发; 执行经验再反过来改进 Skill。

它们不是依次替代的几代技术, 而是围绕大模型逐渐形成的**不同工程层**。

随堂快测

课堂互动 · 3 道题

- Q1 (单选)** 下列哪一项是 Harness 的职责？
A. 决定何时加载某个 Skill, 并执行模型选中的 Tool B. 存储模型的参数
C. 规定 `plugin.json` 的字段 D. 替代 Prompt 直接告诉模型任务
- Q2 (判断)** Agent 通过 Self-learning 生成了一个新 Skill, 说明模型的参数被更新了。
- Q3 (简答)** 实验室助手的 System Prompt 有 3000 token, 其中约八成内容与大多数任务无关。用本讲的词给出一个改造方案 (三步以内)。

答案: Q1 选 **A**; Q2 **✗** 错误, 参数没变, 变的是保存在模型外部、下次重新进入 Context 的经验;

Q3 把各套方法拆成独立 Skill (只留通用要求在 System Prompt) → 靠 description 让 Harness 按任务匹配、渐进式加载 → 打包成 Plugin 供组内分发。

线上提交, 当堂出分。

课后任务与拓展

- 必做① 写一个 Skill: 从“催化裂化开停车”“实验室安全检查”“文献综述”中任选一个, 写出完整的 SKILL.md (frontmatter + 正文), 并列出 references/、assets/、scripts/ 里各应放什么文件 (一页 A4)。
- 必做② 护栏清单: 把互动⑥的讨论整理成 6 条 “Agent 修改 SOP Skill” 的审核规则, 每条标明事前 / 事中 / 事后与负责人 (300 字)。
- 选做③ 装箱: 把①②装进一个 my-lab-plugin/ 目录 (plugin.json + skills/ + mcp.json), 推到 GitHub 公开仓库, 并在期末项目中使用它。

拓展阅读: Skill 开放规范 agentskills.io 与示例仓库
github.com/anthropics/skills;
ReAct 论文 [arXiv 2210.03629](https://arxiv.org/abs/2210.03629); MCP 协议仓库
github.com/modelcontextprotocol (选读)。

Thank
You!

 Your opinion
Matters

第十一讲 · 围绕 Prompt 形成的 Agent 工程体系——Skill、
Harness、Agent Plugins

周天航 · 中国石油大学 (北京)

全部十一讲到此结束。请完成课后任务。